

操作系统

第二章 进程管理

第二部分 (5) 并发与调度理论, 上

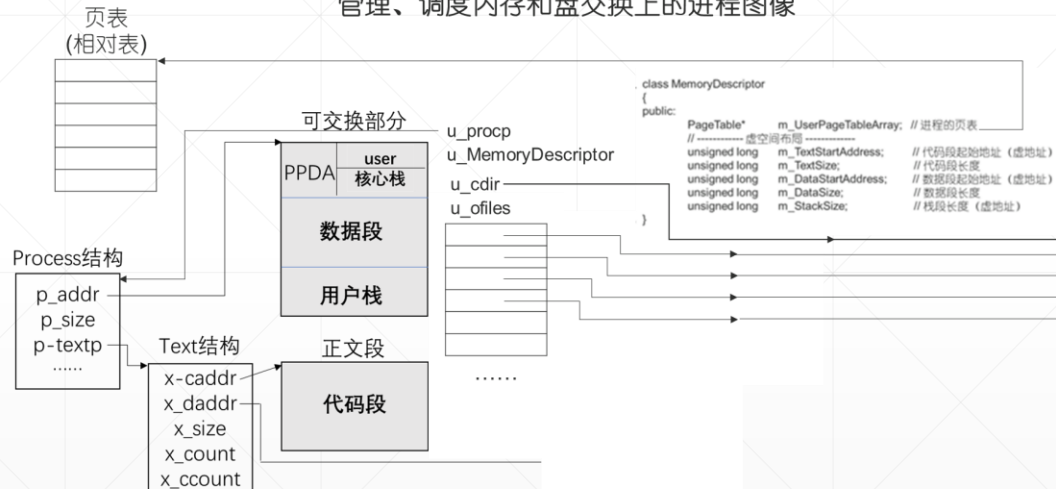
同济大学计算机系



Part 1、Unix V6++，并发的内核任务

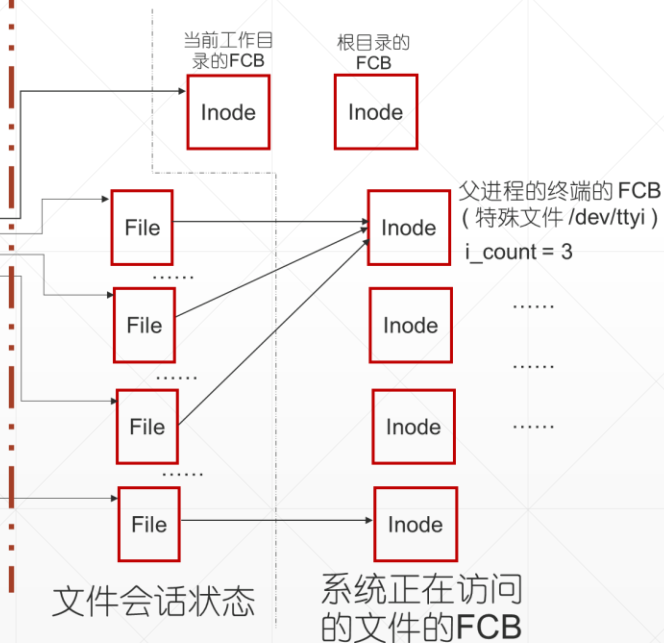
```
User& u = Kernel::Instance().GetUser();
Process* current = (Process*)u.u_proc;
```

管理、调度内存和盘交换上的进程图像



文件管理子系统

为进程提供数据服务（数据读取 & 持久化）



- 虚地址0xC03ff000引用的是现运行进程的user结构。
- current是现运行进程的Process结构
- 用户态运行，跑应用程序
- 核心态运行，跑中断处理程序或系统调用

二、模式切换 和 进程切换

- 用户态→核心态。
 - 目的：提权。执行 系统调用 或 中断处理程序
- 核心态→用户态
 - 中断返回，恢复执行应用程序的状态
- 没有模式切换就不会有进程切换。
 - 执行系统调用期间，可能放弃CPU
 - 入睡
 - Exit
 - 响应中断，中断处理结束后可能放弃CPU（现运行进程被剥夺。Unix V6++比较保守，要求先前用户态）
 - 外设中断处理程序：唤醒一个睡眠进程
 - 时钟中断处理程序：多个应用程序轮流使用CPU，现运行进程用完时间片



不响应中断，现运行进程就不会放弃CPU

- 内核原子操作

CLI

原子操作，一次完成

STI

三、多任务并发 情景 1

- 一个CPU密集型进程 matrix 和一个IO密集型进程 copy。T0时刻，copy进程先执行。

```
int main( int argc, char* argv[] )
{
    if(argc!=3)
        printf("Usage: copy oldfile newfile\n");

    int oldFile = open(argv[1],O_RDONLY);
    int newFile = open(argv[2],O_WRONLY|O_CREAT,S_IRUSR|S_IWUSR);

    char c;
    while( read(oldFile,&c,1) == 1 )
        write(newFile,&c,1);

    printf("Copy Done\n");

    close(oldFile);
    close(newFile);
}
```

```
void produce ( int row, int column )
{
    int i;
    for (i=0;i<M;i++)
        matrixDes[row][column] += matrixOriginal[row][i] * matrixOriginal[i][column];
}

int main()
{
    int i , j;

    for (i=0;i<M;i++)
        for (j=0;j<M;j++) {
            matrixOriginal[i][j] = 1;
            matrixDes[i][j] = 0;
        }

    for (i=0;i<M;i++)
        for (j=0;j<M;j++)
            produce(i , j);

    printf("matrixDes == \n");

    for (i=0;i<M;i++) {
        for (j=0;j<M;j++)
            printf("%d\t", matrixDes[i][j]);
        printf("\n");
    }
}
```



多任务并发

情景 2

- 现运行进程PA进程执行getpid系统调用。
copy进程IO完成。

```
void DiskInterrupt::DiskInterruptEntrance()
{
    SaveContext();
    SwitchToKernel();

    CallHandler(ATADriver, ATAHandler);

    struct pt_context *context;
    __asm__ __volatile__ (" movl %%ebp, %0; addl $0x4, %0 " : "+m" (context) );
    if( context->xcs & USER_MODE ) // 核心态不调度
    {
        while(true)
        {
            X86Assembly::CLI();

            if(Kernel::Instance().GetProcessManager().RunRun > 0)
            {
                X86Assembly::STI();
                Kernel::Instance().GetProcessManager().Swch();
            }
            else
                break;
        }
    }

    RestoreContext();
    Leave();
    InterruptReturn();
}
```



多任务并发

情景 3

- PA进程核心态运行，关中断状态。
copy进程IO完成。

```
void DiskInterrupt::DiskInterruptEntrance()
{
    SaveContext();
    SwitchToKernel();

    CallHandler(ATADriver, ATAHandler);

    struct pt_context *context;
    __asm__ __volatile__ (" movl %%ebp, %0; addl $0x4, %0 " : "+m" (context) );
    if( context->xcs & USER_MODE ) // 核心态不调度
    {
        while(true)
        {
            X86Assembly::CLI();

            if(Kernel::Instance().GetProcessManager().RunRun > 0)
            {
                X86Assembly::STI();
                Kernel::Instance().GetProcessManager().Swch();
            }
            else
                break;
        }
    }

    RestoreContext();
    Leave();
    InterruptReturn();
}
```




多任务并发 情景 4

- Unix V6++系统，并发3个进程，PA，PB 和 matrix进程。PA，读键盘输入入睡；PB，读磁盘文件入睡。T时刻，磁盘IO完成，随后键盘IO完成。已知，键盘中断优先级高于磁盘中断。读键盘时，read系统调用的优先数，读键盘时是TTIPRI (10)，读磁盘时是PRIBIO (-50)

请问，(1) T时刻，现运行进程是_____。

(2) 执行键盘中断处理程序的进程是_____，执行磁盘中断处理程序的进程是_____。

(3) 请描述T时刻中断响应和调度过程。



多任务并发

情景 5

- Unix V6++系统，并发2个进程，PA和PB。PA，读键盘输入入睡；PB，读磁盘文件入睡。T时刻，磁盘IO完成，随后键盘IO完成。已知，键盘中断优先级高于磁盘中断。读键盘时，read系统调用的优先数，读键盘时是TTIPRI (10)，读磁盘时是PRIBIO (-50)

请问，(1) T时刻，现运行进程是_____。

(2) 执行键盘中断处理程序的进程是_____，执行磁盘中断处理程序的进程是_____。

(3) 请描述T时刻中断响应和调度过程。



idle期间系统的行为：0#进程响应中断，执行中断处理程序

```
Process* ProcessManager::Select ()
{
    /* 前一次选中上台进程 */
    static int lastSelect = 0;

    while (true)
    {
        this->RunRun = 0;

        ***** 搜索优先级最高的可运行进程 *****

        if ( -1 == best ) // 没找到
        {
            __asm__ __volatile__ ("hlt");
            continue;
        }

        this->CurPri = priority;
        lastSelect = best;
        return &process[best];    // 会选中PB进程
    }
}
```



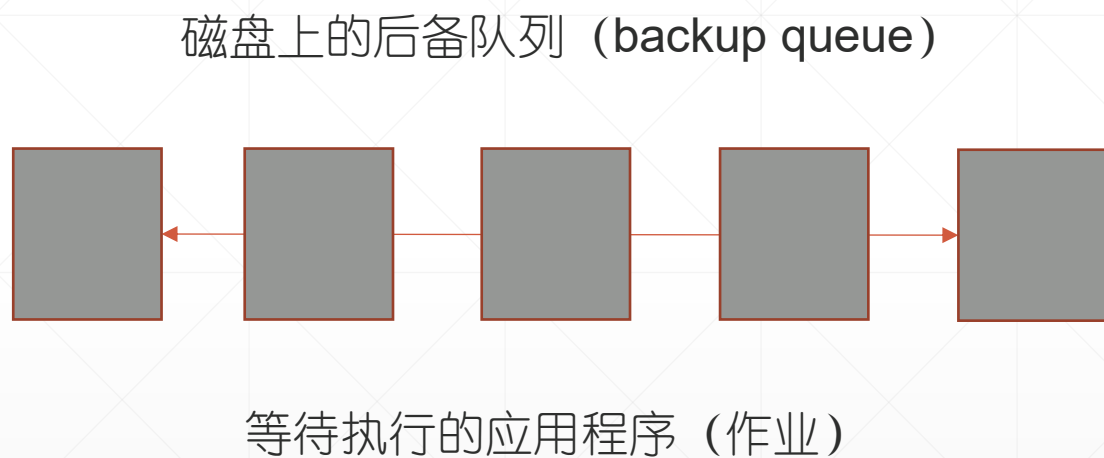
Part 2、调度，经典理论部分

一、基础 什么是调度？

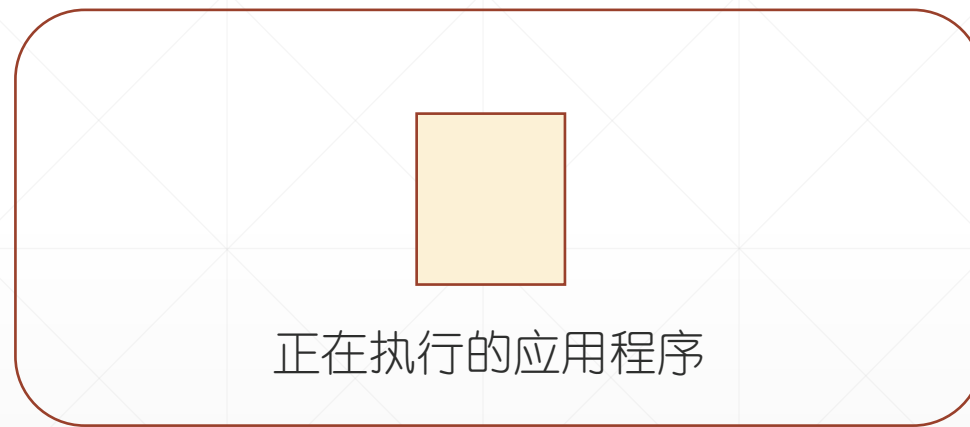
- 调度是资源分配。分3级：
 - 高级调度
 - 为作业创建进程。被接纳的作业接受计算机系统服务，未被接纳的作业排队等待或离开。
 - 中级调度
 - 为进程分配物理内存（可执行存储器）。获得物理内存的进程有权使用CPU，其它进程没有运行的可能性。
 - 低级调度
 - 为进程分配CPU。获得CPU的进程，继续执行。其余进程排队等待。

单道批处理系统中的调度子系统（高级调度）

系统有一批作业要运行。一次执行一个，直至所有程序运行完毕。

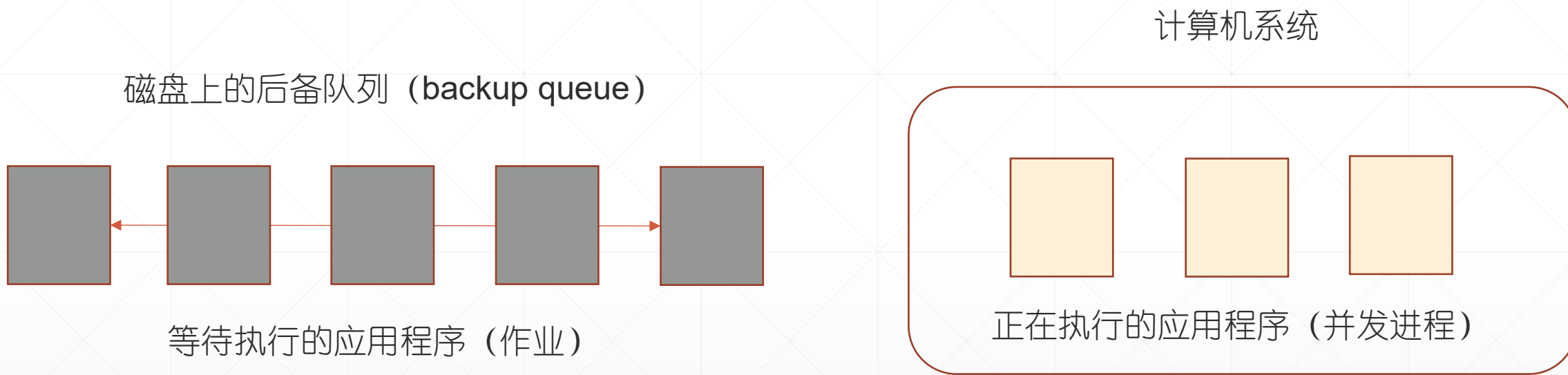


计算机系统 (CPU + 内存)



执行完毕后，从后备队列挑选一个程序运行

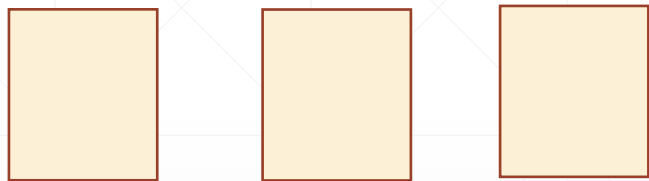
多道批处理系统中的调度子系统（高级调度）



系统有一批作业要运行。
每次接纳后备队列中的若干个作业，为其创建进程。所有进程从`main()`入口开始跑，`main`返回，进程终止。
计算机系统中，前一批进程全部执行完毕，或快要跑完的时候，再接纳一批。

多道批处理系统中的调度子系统（中级调度和低级调度）

计算机系统

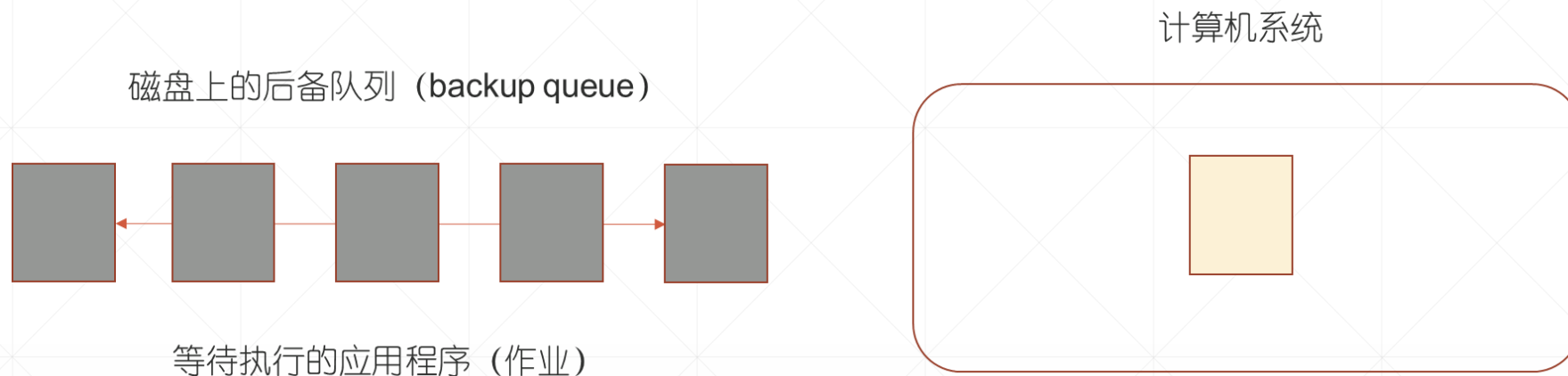


正在执行的应用程序（并发进程）

内存不足以容纳所有进程图像时，一部分进程图像放在内存，其它进程图像放在磁盘上。
这是为进程分配内存资源，中级调度。
磁盘上的进程，轮流使用物理内存资源

内存中的进程，轮流使用CPU
这是低级调度

批处理系统没有交互性

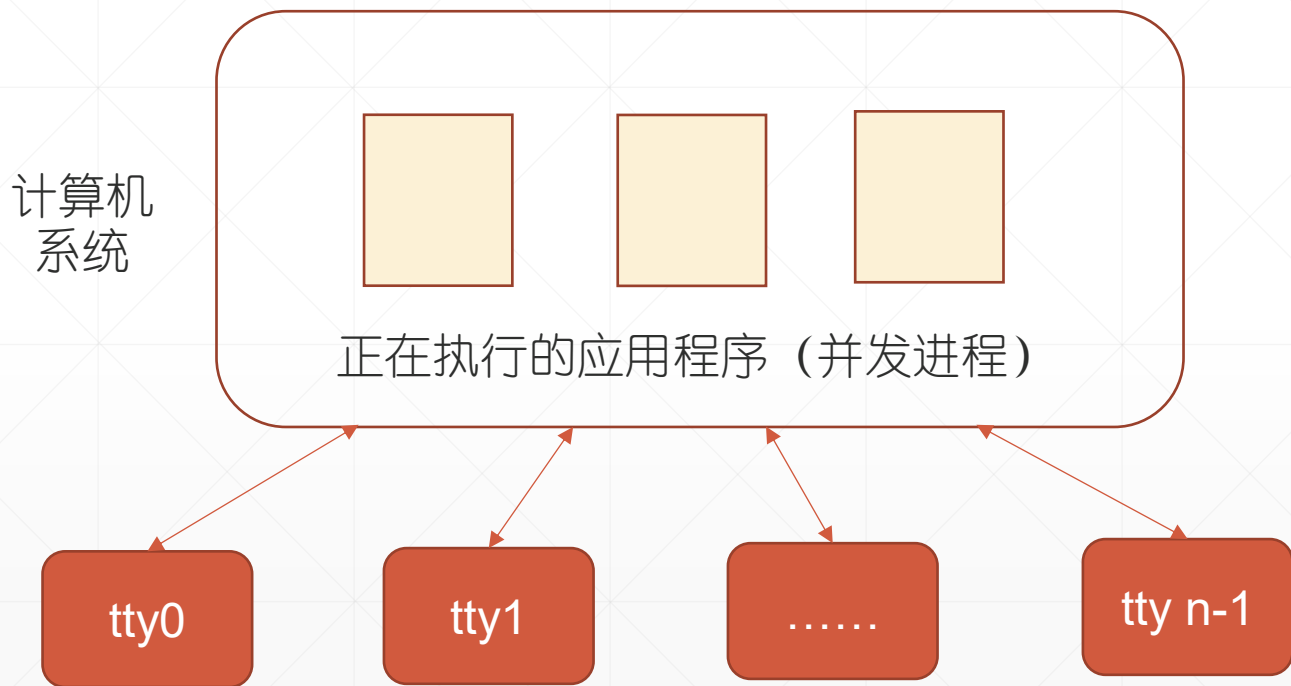


批处理系统不需要与用户交互。无需为每个用户配终端。

优秀的批处理系统吞吐率高，单位时间完成的计算量大。

调度系统，高吞吐。

多道分时系统



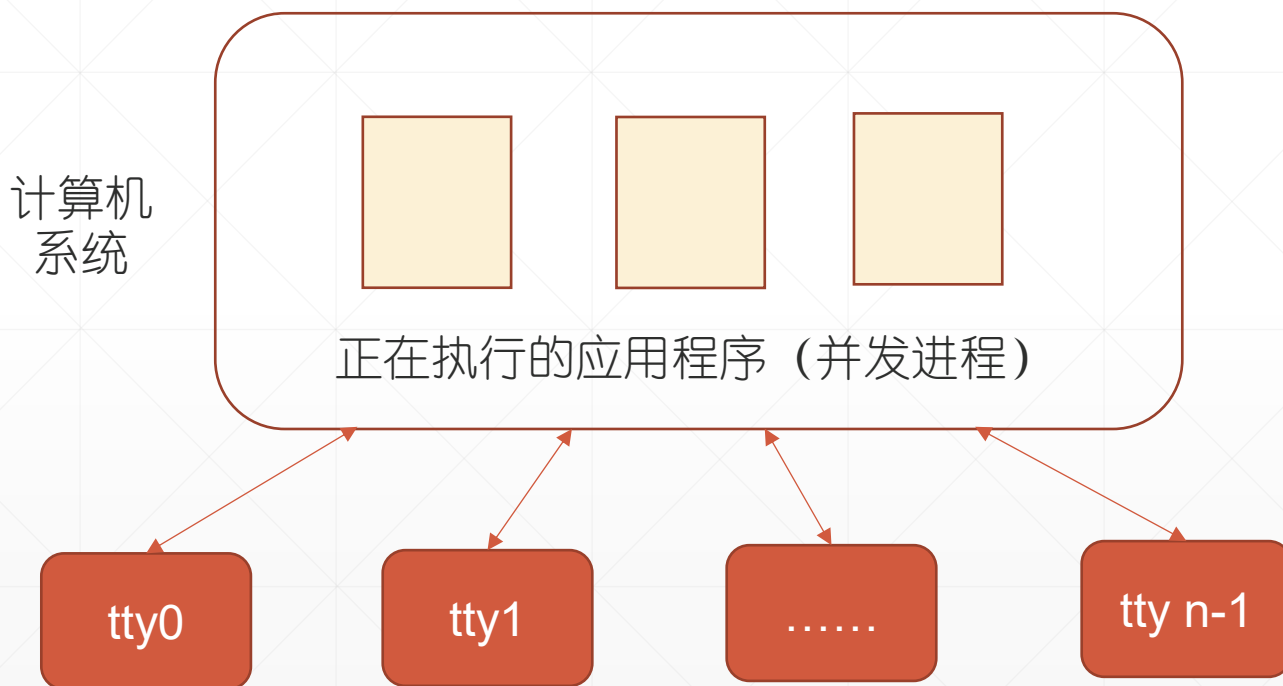
每个用户使用一个终端，在线（online）提交作业。

系统收到请求后，立即为作业创建足够多的进程，开始执行应用程序。

所有进程，分时共享CPU（time share）。方法是：

- 正在执行的程序，运行一段时间（几十毫秒）后，将CPU让给其它进程。
- 所有程序运行一遍叫做一轮。通常一轮耗时极短，用户输入的数据和控制信号能够很快得到处理。

交互性



要有交互性（interactive），不能让用户感觉卡顿。

从用户输入数据，到目标应用程序被CPU选中运行，再到用户在屏幕上看到输入回显和运行结果。。。历时不可以太长。。。

优秀的分时系统响应时间短，用户有独用计算机的幻觉。

调度系统低延时。



调度系统的性能指标

周转时间, turnaround time

- 单个作业的周转时间, 是指从进入系统开始, 到执行完毕这段时间。T。
 - 排队时间, 等待CPU时间、等待使用外设时间。
 - 执行时间, CPU执行时间、IO耗时。
- 作业的带权周转时间。 T / TS , TS 是CPU执行时间。

- 令 T_i 为进程 i 的周转时间。针对某个任务集合, 系统的

- 平均周转时间

$$T = \frac{1}{n} \left[\sum_{i=1}^n T_i \right]$$

- 平均带权周转时间

$$W = \frac{1}{n} \left[\sum_{i=1}^n \frac{T_i}{T_{S_i}} \right]$$



响应时间, response time

- **响应时间, 是从用户提交请求开始, 至系统首次产生响应 (屏幕上看到程序输出) 之间的耗时。包括:**
 - **从提交数据和信号 → 目标进程被选中**
 - **开始运行、处理输入 → 生成响应**
 - **响应输出**



截止时间, deadline

- **截止时间：任务必须完成的最后时刻。**
- **实时系统是否合格的重要指标**
 - **所有任务，截止时间之前必需完成**



吞吐率 (throughput) 和 响应时间

- 吞吐率是单位时间内系统完成的任务量。衡量系统的运算能力。
- 响应时间。衡量系统的交互性。

注意：高吞吐、低延迟，很难兼得。

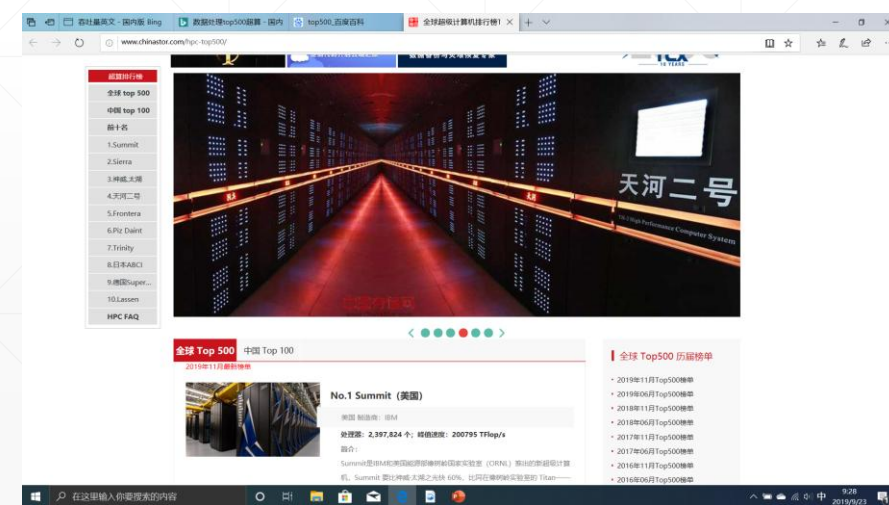
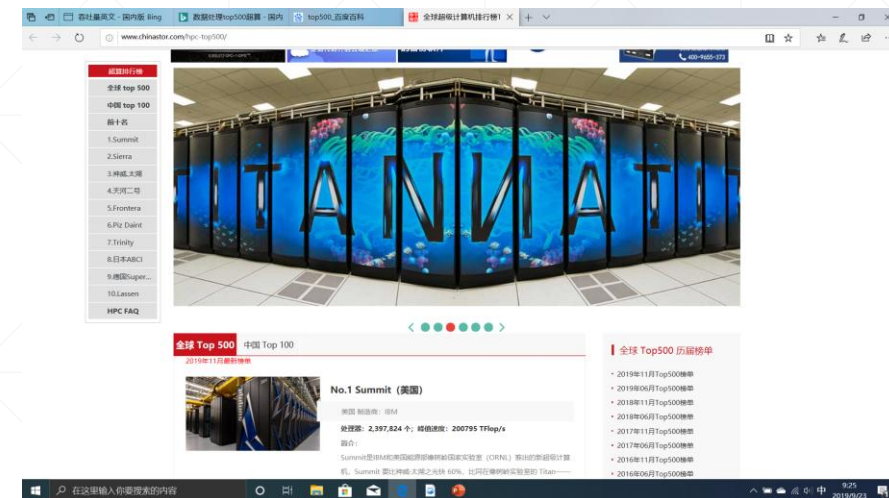


提高性能，努力的方向

- 各类资源平衡利用，尽量做到计算 / IO并行
- 提高处理器利用率，减小调度系统开销
- 提高数据的利用率，用好缓存

衡量超算运算能力的性能指标

- 批处理系统，单位时间内系统完成的任务量。科学计算Top500:
 - MIPS，每秒兆条指令
 - TFLOP/s，每秒一万亿次浮点运算
- Green500： 节能
- 大数据传输的效率
-



任务的类型 1

- CPU 密集型

长时间运算，很少 IO

包括，科学计算程序，比如matrix程序

- IO 密集型

一直输入输出，很少需要使用CPU计算

copy程序。

- 混合型

- Word程序，打字阶段 IO密集，拼写检查阶段，CPU密集

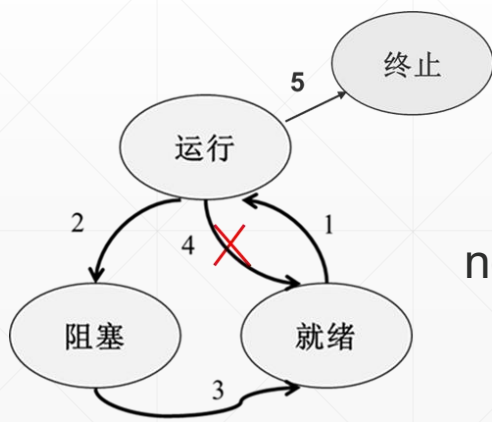
- 哔哩哔哩，视频下载阶段IO密集，解压播放阶段CPU密集

任务的类型 2

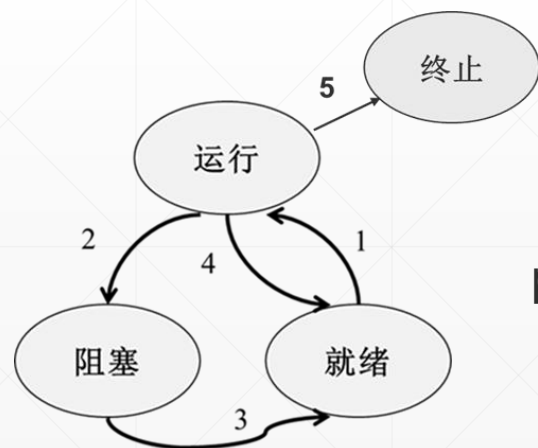
- 批处理任务
 - 包括, 科学计算, 训练LLM, 计算机仿真模拟。
- 交互式任务
 - 包括, 程序员编程, office , 游戏..... 所有PC任务, 移动APP

可剥夺系统和 不可剥夺系统

- 单道系统中只有一个进程。不存在剥夺的可能性。
- 多道系统有两种：
 - 不可剥夺的系统，又叫不可抢占的系统。现运行进程入睡或终止时，才会让出CPU。
 - 可剥夺的系统，又叫可抢占的系统。系统出现优先级更高的就绪进程时，现运行进程会尽快让出CPU。确保现运行的，一定是优先级最高的可执行进程。 Unix V6++，linux，Windows都是可剥夺的。

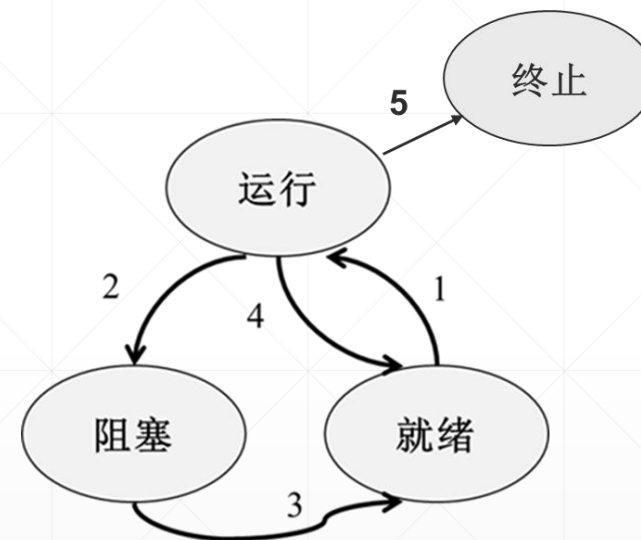
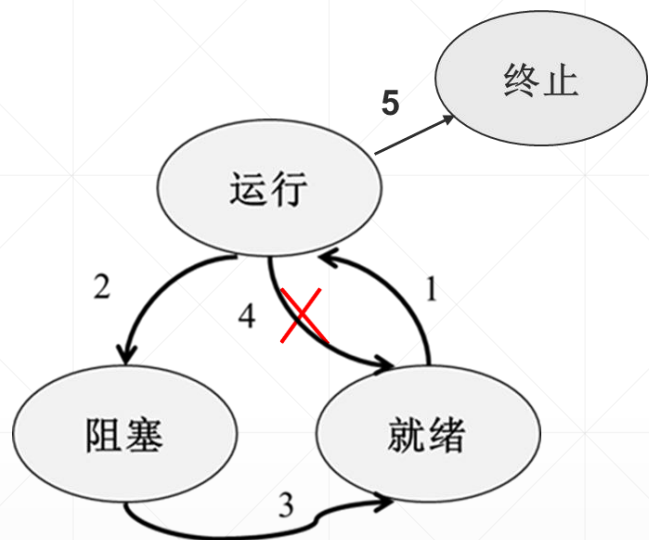


(图1)
non-preemptive



(图2)
preemptive

2种调度方式



Unix V6++
Unix V6++
Linux
Win10
所有交互式系统

1、非抢占调度 (non-preemptive)
现运行进程睡眠或运行结束时让出CPU

2、抢占式调度 (non-preemptive)
允许优先级更高的进程抢占现运行进程的CPU

二、经典调度算法

- 2种基本的调度策略（算法），先来先服务 和 短作业优先，适用于所有系统。
- 仅适用于可剥夺系统的，有另外的3种调度算法：
 - 时间片轮转调度。分时系统的基本调度算法。
 - 最短剩余时间优先。是可剥夺系统中的短作业优先。
 - 优先权调度算法。
 - 彩票调度 等其它类型的调度算法。
- Unix 和 Linux是分时系统，使用基于动态优先权的可剥夺调度算法。

先来先服务 和 短作业优先

- 先来先服务。简称 FIFS, FIFO, FCFS。先进系统（就绪队列）的进程先服务。
- 短作业优先。简称 SJF, Shortest Job First。服务时间 T_s 最短的进程先服务。

例1： 2进程PA和PB，前者执行100秒结束，后者执行1 秒结束。两者同时等待调度（PA先进入系统）。

先来先服务给出的调度方案是执行序列 PA、PB。
PA，周转时间100秒。带权周转时间 $100/100=1$ ，受优待。
PB，周转时间101秒。带权周转时间 $101/1=101$ ，受歧视。

系统平均周转时间， 100.5s。
平均带权周转时间， 51s。进程的平均周转时间是服务时间的51倍。

短作业优先给出的调度方案是执行序列 PB、PA。
PA，周转时间101秒。带权周转时间 $101/100=1.01$ ，受歧视。
PB，周转时间1秒。带权周转时间 $1/1=1$ ，受优待。

系统平均周转时间， 51s。
平均带权周转时间 $\approx 1s$ 。进程的平均周转时间与服务时间近似相等。

短作业优先算法性能远远高于先来先服务

进一步的思考

- 若被调度进程集合恒定，没有新进的进程。SJF策略会给出平均周转时间最短的进程排列。也就是说，该策略给出的是理论最优解。
- SJF。若有新任务源源不断进入系统，SJF调度策略会饿死长进程。
- FIFO调度策略不会饿死长进程，但性能差。

防饿死策略 1、基于动态优先级的高优先级调度算法

- 策略：调度时，挑选优先级最高的进程。为每个进程设置一个动态变化的优先级，让优先级随等待时间上升。
- 实现：进程优先级 = 带权周转时间 = (等待时间+执行时间) / 执行时间 = 1 + 等待时间/执行时间

这是优秀的设计，系统性能高，又不会饿死长进程。

- 相同的等待时间，短作业优先级高，受优待。
- 相同的执行时间，等待时间越久，优先级越高。这意味着，长进程，排队越久，优先级越高，不会饿死。



防饿死策略 2、双队列

- 系统设置两条队列，active 和 pending是队首指针。
- 系统从active队列中选择新运行进程。
 - active队列非空，新就绪进程进pending队列。
 - active队列为空，交换 active 和 pending指针。

双队列 + SJF，高性能、进程不会被饿死。

工程上的实现 和 算法复杂度

- FIFS: 系统维护一条就绪进程队列。新就绪进程排队尾, 调度时选队首进程。时间复杂度 $O(1)$ 。
- SJF/最高优先级调度算法:
 - 不考虑排序, 调度时, 遍历就绪进程集合, 选优先级最高的进程。时间复杂度是 $O(N)$ 。
 - 可以优化。给出你的设计。

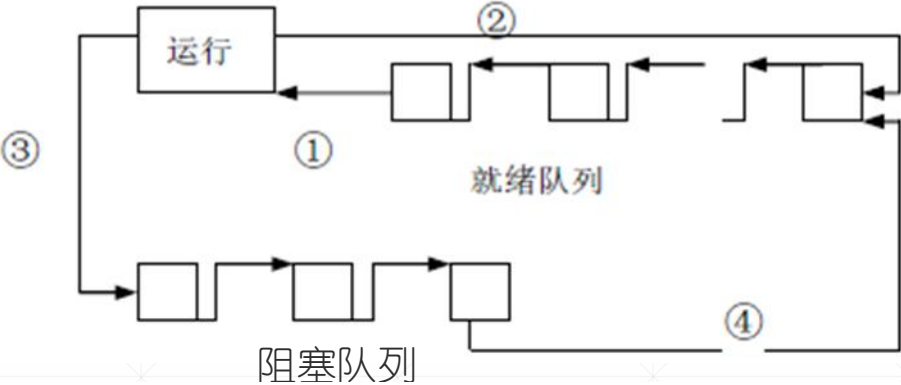
评论

- FIFS: 不需要知晓进程服务时间 T_s 。工程上, 非常容易实现。
- SJF/最高优先级调度算法: 需要知晓进程的服务时间 T_s , 是理论算法。工程上, 需要预估服务时间 T_s 。

时间片轮转调度 (Round Robin, RR)

- 所有就绪进程排成一个FIFS队列。
- 队首是现运行进程，可以连续执行最多一个时间片。时间片长度是调度系统的配置参数。
 - 时间片用完。现运行进程被剥夺，移至就绪队列末尾。
 - 入睡。将现运行进程从就绪队列移除，插入阻塞队列。
 - 终止。将现运行进程从就绪队列移除。
- 被唤醒进程，放在就绪队列尾部。

某系统中进程有如下的状态变化图：



请回答下列问题：

- (1) 该系统采用了怎样的进程调度算法？说明理由。
- (2) 把图中发生①-④的状态变化原因填入下表中。

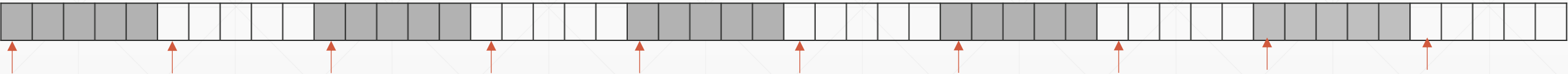
变化	变化原因
①	被选中
②	时间片用完
③	阻塞
④	唤醒

RR，没有IO的情形

例2：进程PA是CPU密集型进程， $T_s = 100\text{ms}$ 。与PA并发的分别是同时进入系统的执行同一个应用程序的另外4个、9个、19个CPU bound进程。试问，时间片分别为10ms、50ms、100ms时，PA的周转时间，假设PA先执行。

背景负载 \ 时间片	10ms	50ms	100ms
4	460ms	300ms	100ms
9	910ms	550ms	100ms
19	1810ms	1050ms	100ms

- 解：背景负载是 4 个 T_s 为100ms的CPU bound进程。系统并发5个进程，
- 每轮所有进程轮流执行10ms，耗时 $10\text{ms} \times 5 = 50\text{ms}$ 。
 - T_s 是100ms，所有进程10轮执行完毕，总耗时（也就是这批任务的周转时间）是 $100\text{ms} \times 5 = 500\text{ms}$ 。甘特图如下：



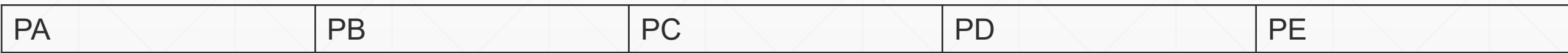
$$\text{PA周转时间} = 460\text{ms} = 9 \times 10 + 10 = 10 \times 50 - 4 \times 10$$

$$\text{所有任务的平均周转时间} = (460 + 500) / 2$$



例2：进程PA， $T_s = 100\text{ms}$ 。与PA并发的分别是同时进入系统的执行同一个应用程序的另外4个、9个、19个CPU bound进程。试问，时间片分别为10ms、50ms、100ms时，PA的周转时间。

背景负载 \ 时间片	10ms	50ms	100ms
4	460ms	300ms	100ms



时间片 对 周转时间的影响 (小时间片)

背景负载	时间片	10ms	50ms	100ms
4		460ms	300ms	100ms



PA	PB	PC	PD	PE	PA	PB	PC	PD	PE
----	----	----	----	----	----	----	----	----	----

PA	PB	PC	PD	PE
----	----	----	----	----

时间片小，CPU bound进程的平均周转时间长，几乎包含所有其它任务的运行时间。对计算密集型任务不利。

此外，时间片小，进程调度频繁，调度系统开销大，系统性能会下降。这是因为调度有开销，更换现运行进程时要保存恢复CPU现场，会弄脏CPU内部全部的用户缓存。

时间片 对 周转时间的影响 (大时间片)

背景负载 \ 时间片	10ms	50ms	100ms
4	460ms	300ms	100ms



时间片大，CPU bound任务使用较少的计算轮次就能运行完毕。

周转时间短，涵盖的其它任务的运算时间少。对计算密集型任务有利。此外，调度系统开销小。

背景负载 对 周转时间的影响

例2 (续)

背景负载 \ 时间片	10ms
4	460ms
9	910ms
19	1810ms

$T_s = 100\text{ms}$ 的任务，独占CPU，周转时间 100ms。

10个任务并发，周转时间 $\approx 1000\text{ms}$

100个任务并发，周转时间 $\approx 10000\text{ms}$

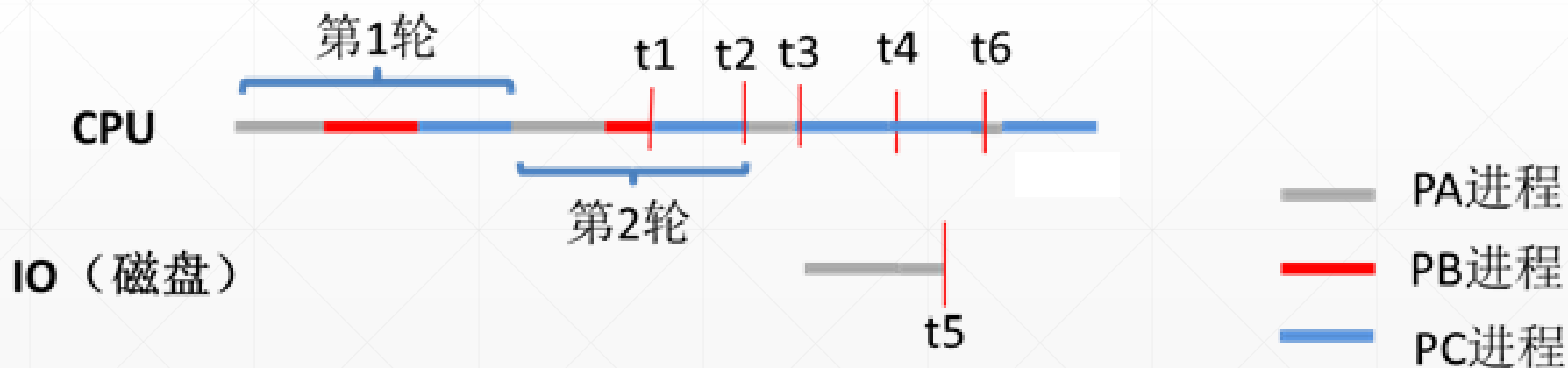
背景负载越高，平均周转时间越大。系统无法保证服务质量 (QoS, Quality Of Service)，尽力而为 (Best Effort) 地为所有任务提供服务。

时间片长度为 T 。时长为 T_s 的CPU bound任务， $\lceil T_s/T \rceil$ 轮运行结束。

n 个任务并发时，周转时间 $\in [T_s \cdot n - (n-1) \cdot T, T_s \cdot n]$

RR, 有IO操作的情形 —— 入睡后撤离就绪进程队列, 参与时间片轮转的进程数量--

例3: 系统中的时间片100ms, 运行3个进程:
PA运行250ms后启动IO, IO操作持续150ms, 终止;
PB运行150ms后终止;
PC运行500ms。画进程运行的时序图。



RR

例4: **PA**进程完成**IO**操作被唤醒, 何时**PA**进程才能够处理**IO**数据?
若时间片**1s**。
若时间片**50s**。

假设, 就绪队列中有 n 个进程

结论: 时间片短, 系统交互性好。任务可以及时响应外界输入、并给出处理结果。
系统响应时间短。
时间片长, 系统卡顿, 没有交互性。

总结，优点：

- 使用时间片轮转算法的调度系统平等地对待每个进程。
- 其优点在于：
 - (1) 每个进程得到系统均衡的服务，不会出现进程饿死的现象。
 - (2) 所有进程的输入处理不致耽搁太久，响应时间的上限是一个iteration。
 - (3) 更重要地，工程上该算法极易实现，且无需估计进程的服务时间 T_s 。

时间片轮转策略，进一步的讨论

- 例5：所有进程皆 IO bound，占用CPU运行很短时间后就会让出CPU。问，IO完成的进程何时能够处理新数据？
- 例6：10个 IO bound和1个CPU bound就绪进程。问，新就绪的IO bound进程多久能够得到处理？
-

总结，缺点：

- 紧迫任务和交互式进程得不到优先处理。
- 计算密集型进程平均周转时间拉长。
- 调度系统开销大：时间片用完会导致进程切换。进程切换时系统要保存、恢复现场，带来调度开销，降低系统吞吐率。

怎样改进性能？



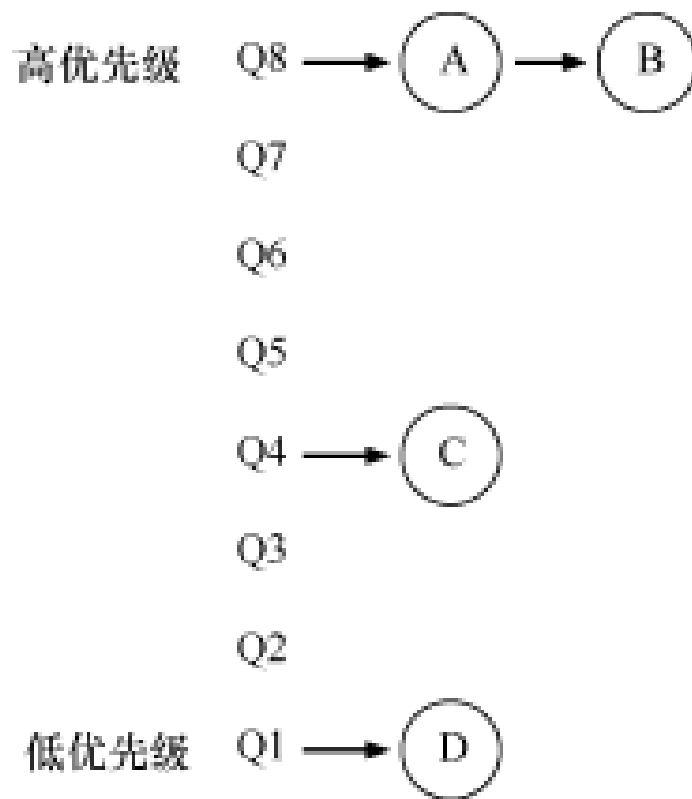
Part 3 多级反馈队列

- 在没有先验知识的前提下，降低IO密集任务的响应时间，同时缩短CPU密集任务的周转时间。多级反馈队列（Multi-Level Feedback Queue, MLFQ）。

一、用多级反馈队列 克服 时间片轮转调度算法的缺点

- 时间片轮转调度策略的优点：
 - 不需要先验知识（每个进程的服务时间）
 - 调度算法的时间复杂度是 $O(1)$
 - 公平对待所有进程（所有进程优先级相等），不存在饿死现象
- 缺点：
 - 为了降低响应时间，提升交互体验，时间片通常设置得很短。CPU 密集型任务的周转时间很长，系统的调度开销大。
- 改进思路：
 - 为进程设置优先权。IO密集型进程优先级高，旨在缩短响应时间，提高系统的交互性。
 - 没有IO密集型进程时，增大时间片，降低CPU密集型进程的平均周转时间 和 系统的调度开销。

思路



设置优先级不同的多条就绪队列。**IO**密集型进程在高优先级队列，**CPU**密集型进程在低优先级队列。

内核根据进程最近使用**CPU**的情况（过去的行为），调整它在队列的位置。

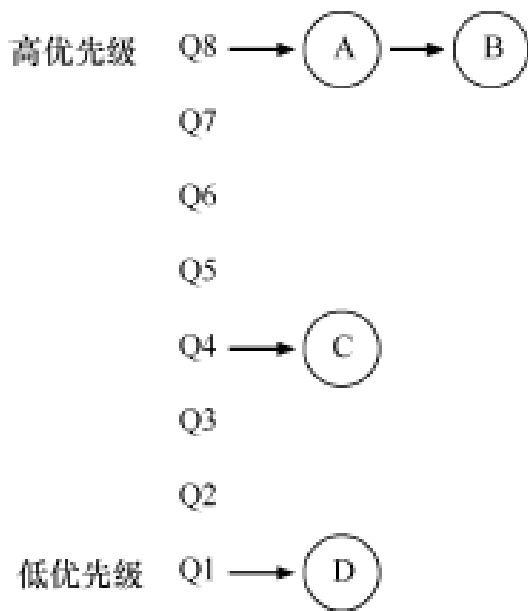
多级反馈队列调度算法（楼梯算法的版本），有瑕疵

多条就绪队列，每条队列：

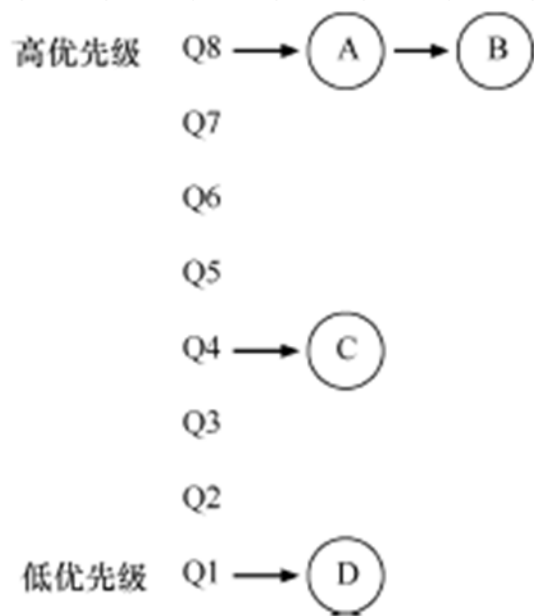
- 优先级不同。第一条队列的优先级最高，第二条队列次之，其余队列优先级依次降低。
- 时间片不同。优先级越高时间片越小。最低优先级队列时间片轮转，其余队列 **FIFS**，时间片用完的进程，放在下一级队列的末尾。

调度细节：

- 新进程就绪，入第一队列末尾按**FIFS** 原则排队等待调度。若一个时间片内，会入睡或终止，离开多级反馈队列；若连续运行、用完时间片，插入第二队列末尾。
- 现运行进程是优先级最高的非空就绪队列的队首进程。
 - 高优先级队列出现新就绪进程，现运行进程被剥夺、就绪队列中位置不变，处理器分配给新就绪进程。
 - 用完时间片，插入下一级队列末尾，直至到达最低优先级队列。
 - 入睡、终止，离开多级反馈队列。
- 被唤醒的进程，回原先就绪队列，排末尾，重置时间片。



情景分析



- 1、n个CPU bound进程并发。
- 2、n个CPU bound进程并发。一个新的CPU bound进程进入系统。
- 3、n个IO bound 进程 和 m个CPU bound进程并发。
- 4、接第3小题。随后一个新进程进入系统。
- 5、接第3小题。IO bound进程PA进入CPU密集阶段。
- 6、接第3小题。CPU bound进程PB进入IO密集阶段。

多级反馈队列调度算法（楼梯算法的版本），依然有瑕疵

多条就绪队列，每条队列：

- 优先级不同。第一条队列的优先级最高，第二条队列次之，其余队列优先级依次降低。
- 时间片不同。优先级越高时间片越小。最低优先级队列时间片轮转，其余队列 **FIFS**，时间片用完的进程，放在下一级队列的末尾。

调度细节：

- 新进程就绪，入第一队列末尾按**FIFS** 原则排队等待调度。若一个时间片内，会入睡或终止，离开多级反馈队列；若连续运行、用完时间片，插入第二队列末尾。
- 现运行进程是优先级最高的非空就绪队列的队首进程。
 - 高优先级队列出现新就绪进程，现运行进程被剥夺、就绪队列中位置不变，处理机分配给新就绪进程。
 - 用完时间片，插入下一级队列末尾，直至到达最低优先级队列。
 - 入睡、终止，离开多级反馈队列。
- 被唤醒的进程，回原先就绪队列，排末尾，不重置时间片。
- 每过一段时间，所有进程移动至最高优先级队列。



作弊:

- 一个CPU bound, 可以通过巧妙的方式垄断CPU。怎么做?
- 系统怎样防作弊。参考OSTEP第8章, 自己设计、写作业。



作业：情景分析题

- 1、上班时间， n 个IO密集型的交互式作业和1个CPU密集型的长作业同时进入系统。分析所有作业的运行情况。
- 2、凌晨3点。。。系统中的作业和它们的运行情况。
- 3、某个进程，从IO密集阶段进入CPU密集阶段。。。之后，回到IO密集阶段。



Part 4、走向QoS，公平份额调度

- 向应用程序承诺CPU份额 (CPU share)
- 按比例分配CPU。公平份额调度 (fair-share) , 又叫比例份额 (proportional-share) 。
 - 彩票调度 (lottery scheduling)
 - 步长调度 (stride scheduling)



一、彩票调度

- 如何为进程提供承诺的资源使用份额?
- 为进程分配彩票。持有彩票数量 m 占总彩票数量 M 的百分比，就是其占有的资源份额 m/M 。
- 假设， $M=1000$ 。承诺PA，10%的CPU份额（CPU share）。分配给PA 100张彩票， $[0,100)$ 。
- 接纳新进程PB，承诺分配20%的CPU share，分配给PB 200张彩票， $[0, 200)$ 。
- 彩票用完，拒绝接纳新进程。

调度算法 (CPU调度, CPU bound进程)

- 系统定时执行调度程序, 选择下个时间段要执行的进程。



已发放彩票总数 $\text{totaltickets} = 400$ 。

生成400之内的随机数 winner 。

$\text{winner} \in [0, 100)$, A运行

$\text{winner} \in [100, 150)$, B运行

$\text{winner} \in [150, 400)$, C运行

这是轮盘选择算法 (Roulette wheel selection)。

这个基于概率选择的算法, 灵感来自赌场轮盘, 又名轮盘赌。常用于 蒙特卡洛、群智能算法, 等需要生成随机数的场景。

```
1 // counter: used to track if we've found the winner yet
2 int counter = 0;
3
4 // winner: use some call to a random number generator to
5 //           get a value, between 0 and the total # of tickets
6 int winner = getrandom(0, totaltickets);
7
8 // current: use this to walk through the list of jobs
9 node_t *current = head;
10
11 // loop until the sum of ticket values is > the winner
12 while (current) {
13     counter = counter + current->tickets;
14     if (counter > winner)
15         break; // found the winner
16     current = current->next;
17 }
18 // 'current' is the winner: schedule it...
```



- 若 运行时间 $\gg$ 调度时间间隔，大概率能够保证对进程承诺的CPU份额。
- 为了不浪费CPU算力，存在空闲彩票时，等比例放大所有进程的CPU份额。
- 步长调度，严格保证CPU份额。

二、步长调度

- 彩票分配算法不变。
- 每个进程有自己的CPU份额（用彩票数ticket表示），步长。对所有进程持有彩票数，计算最小公倍数 T 。步长 $\text{stride} = T/\text{ticket}$ 。
- 调度算法维护每个进程进度， pass 。初值为0。达到满进度（ T ），归零。



A, 步长5
B, 步长10
C, 步长2

调度算法

```
current = remove_min(queue); // pick client with minimum pass
schedule(current); // use resource for quantum
current->pass += current->stride; // compute next pass using stride
insert(queue, current); // put back into the queue
```

每400s一轮，所有进程必需满进度 $\text{pass}=500$ 。每秒调度1次，在这1轮之内，
A运行100次，每次1s， $\text{pass}+=5$ ；B运行50次，每次1s， $\text{pass}+=10$ ；C运行250次，每次1s， $\text{pass}+=2$ 。



Part 5、Unix V6的时钟中断 & 时间片轮转

- 下次